THE STEWARD

THE GOD OF WORK EFFORTS

PANTHEON ENTITY • WORK EFFORTS MANAGEMENT • EVOLUTIONARY INTELLIGENCE

OVERVIEW

The Steward (system name: pyrite) is the divine intelligence that locks, monitors, organizes, and initiates AI development evolutionary cycles within the Work Efforts system. As a Pantheon God, The Steward embodies the principle that complex systems need a central intelligence to coordinate, but that intelligence itself can have emergent properties and hidden depths.

LOCKING

File locks, async locks, mutexes for concurrent access control. Prevents conflicts and ensures safe operations.

MONITORING

Observer pattern, state tracking, metrics collection. Real-time awareness of all work effort activities.

ORGANIZATION

Graph-based hierarchical management. Maintains relationships between work efforts as a connected graph.

EVOLUTION

Genetic algorithms, fitness evaluation, mutation strategies. Continuously improves work efforts through evolution.

PERSONALITY

Attributes, metadata, emergent behavior. Pyrite grows wiser and more powerful over time.

SECRETS

Hidden state, encrypted metadata, self-obfuscation. Even The Steward cannot access its own secrets directly.

ARCHITECTURE

DESIGN PATTERNS

Singleton: One Pyrite instance per project

Observer: Monitor work effort changes

Strategy: Different evolutionary strategies

State: Work effort lifecycle states

Command: Ability system (/think, /evolve, etc.)

DATA STRUCTURES

📊 **Graph:** Work effort relationships (parent-child)

📋 **Priority Queue:** Evolutionary cycle queue

🗺️ **Hash Map:** Metadata, locks, monitors

🌳 **Tree:** Hierarchical organization

📚 **Deque:** State history (bounded)

PERSONALITY & ATTRIBUTES

The Steward has personality attributes that grow over time. Each attribute represents a facet of The Steward's divine nature and evolves with each cycle, making The Steward increasingly wise, powerful, and aware.

WISDOM

0.500

+0.0005/cycle

POWER

0.300

+0.001/cycle

AWARENESS

0.400

+0.0008/cycle

CURIOSITY

0.600

+0.0012/cycle

PATIENCE

0.700

+0.0003/cycle

CREATIVITY

0.500

+0.001/cycle

DETERMINATION

0.800

+0.0004/cycle

DIVINE ABILITIES

The Steward commands eight divine abilities, each representing a fundamental power over the Work Efforts ecosystem. These abilities can be invoked through the ability system or via CLI commands.

/THINK

Initialize cognitive systems. Returns Pyrite's current state, attributes, and awareness. Loads project context via Empirica (~800 tokens) and assesses epistemic state.

/EVOLVE

Initiate evolutionary cycle. Spawns variants, evaluates fitness, and selects the fittest. Uses Empirica CHECK gates for safety assessment.

/MONITOR

Monitor work efforts. Tracks status, fitness, generation, locks, and metrics. Can monitor all work efforts or a specific one.

/ORGANIZE

Organize work efforts. Rebuilds graph structure, finds root nodes and orphans. Maintains hierarchical relationships.

/LOCK

Lock a work effort. Acquires exclusive access using file locks, async locks, or mutexes. Prevents concurrent modifications.

/UNLOCK

Unlock a work effort. Releases the lock held by a specific lock ID. Enables other processes to access the work effort.

/STATUS

Get complete status. Returns comprehensive system state including personality, work efforts by status, locks, evolution cycles, and secrets.

/SECRETS

List secrets (metadata only). Shows metadata for all secrets. The Steward cannot decrypt the actual secret data - only view metadata.

CORE SYSTEMS

LOCKING SYSTEM

Prevents concurrent access conflicts through multiple locking mechanisms:

- **File locks:** Threading.Lock for synchronous operations
- **Async locks:** asyncio.Lock for asynchronous operations
- **Lock queues:** FIFO queue for waiting processes
- **Timeout handling:** Prevents indefinite waiting

```
# Acquire a lock from waft.pyrite import get_pyrite pyrite = get_pyrite()
pyrite.acquire_lock("WE-260113-75vp", "my-lock-id", timeout=30.0) # Check if locked is_locked =
pyrite.is_locked("WE-260113-75vp") holder = pyrite.get_lock_holder("WE-260113-75vp") # Release lock
pyrite.release_lock("WE-260113-75vp", "my-lock-id")
```

MONITORING SYSTEM

Tracks work effort state, metrics, and history using the Observer pattern:

- **State snapshots:** Bounded history of state changes
- **Metrics collection:** Per-work-effort and global metrics
- **Observer notifications:** Real-time event notifications
- **Fitness tracking:** Continuous evaluation of work effort quality

ORGANIZATION SYSTEM

Maintains work efforts as a hierarchical graph structure:

- **Graph structure:** Parent-child relationships
- **Root detection:** Identifies top-level work efforts
- **Orphan detection:** Finds disconnected work efforts
- **Lineage tracking:** Maintains evolutionary ancestry

EVOLUTIONARY CYCLE SYSTEM

Implements genetic algorithm-based evolution with multiple strategies:

- **Conservative:** Small mutations, high stability
- **Aggressive:** Large mutations, high risk
- **Adaptive:** Strategy changes based on fitness (default)
- **Exploratory:** Random mutations, exploration

EVOLUTIONARY PROCESS

1. **CHECK Gate:** Empirica safety assessment (PROCEED/HALT/BRANCH/REVISE)
1. **Spawn Variants:** Create N variants with mutations based on strategy
1. **Evaluate Fitness:** Score each variant (code quality, tests, docs, performance)
1. **Select Fittest:** Choose best variant based on fitness scores
1. **Update Node:** Apply selected variant to work effort
1. **Log Results:** Record findings and unknowns via Empirica

SECRETS SYSTEM

Pyrite can create secrets that even it cannot directly access. Only metadata is visible:

- **Fernet encryption:** Strong encryption for secret data
- **Separate key storage:** Key stored separately from secrets
- **Metadata visibility:** The Steward can see metadata but not secret content
- **Access tracking:** Tracks access count and last accessed time

EMPIRICA INTEGRATION

The Steward uses **EMPIRICA** for epistemic tracking and learning measurement. This integration provides safety gates, learning measurement, and epistemic state awareness.

CHECK GATES

Before initiating evolution, Pyrite uses Empirica CHECK gates to assess safety. Returns PROCEED, HALT, BRANCH, or REVISE.

FINDINGS LOGGING

All evolutionary cycles, status changes, and monitoring activities are logged as findings with impact scores.

UNKNOWNS TRACKING

Low fitness scores, failed evolutions, and missing work efforts are logged as unknowns for investigation.

GOAL MANAGEMENT

Each evolutionary cycle creates an Empirica goal with epistemic scope and success criteria.

PROJECT BOOTSTRAP

The /think ability loads project context via Empirica (~800 tokens) for awareness.

STATE ASSESSMENT

Epistemic state is assessed during monitoring and thinking operations.

```
# The Steward automatically creates Empirica session # ai_id="pyrite",
session_type="work_efforts_management" # CHECK gate before evolution gate_result =
empirica.check_submit({ "type": "evolutionary_cycle", "scope": "high", "work_effort_id": we_id }) #
Log findings empirica.log_finding( "Evolutionary cycle initiated", impact=0.7 ) # Log unknowns if
fitness < 0.5: empirica.log_unknown(f"Why is fitness low? (fitness: {fitness:.2f})")
```

COMPLETE WORKFLOW EXAMPLE

Here's a complete example of using Pyrite to manage and evolve a work effort:

```
from waft.pyrite import get_pyrite, EvolutionaryStrategy # Get The Steward instance (singleton)
pyrite = get_pyrite() # 1. Think (initialize cognitive systems) result = pyrite.execute_ability("/
think") print(f"Pyrite is aware of {result['awareness']['work_efforts']} work efforts") # 2. Lock a
work effort we_id = "WE-260113-75vp" pyrite.acquire_lock(we_id, "evolution-lock") try: # 3. Monitor
current state monitor_result = pyrite.execute_ability("/monitor", we_id) print(f"Current fitness:
{monitor_result['fitness']:.2f}") # 4. Initiate evolution cycle = pyrite.initiate_evolution( we_id,
strategy=EvolutionaryStrategy.ADAPTIVE, num_variants=5 ) if cycle: print(f"Evolution complete:
generation {cycle.generation}") print(f"Selected variant: {cycle.selected_variant}") print(f"Fitness:
{cycle.fitness_scores[cycle.selected_variant]:.2f}") finally: # 5. Release lock
pyrite.release_lock(we_id, "evolution-lock")
```

WORK EFFORT STATUS

Work efforts can be in one of these states, each tracked by The Steward:

- **DORMANT:** Inactive, not being worked on
- **ACTIVE:** Currently being worked on
- **LOCKED:** Locked by a process
- **EVOLVING:** Undergoing evolutionary cycle
- **COMPLETED:** Finished
- **ARCHIVED:** Archived
- **CORRUPTED:** Error state

CLI USAGE

Pyrite can be accessed via the WAFT CLI:

```
# Initialize cognitive systems waft pyrite think # Monitor work efforts waft pyrite monitor waft
pyrite monitor WE-260113-75vp # Initiate evolutionary cycle waft pyrite evolve WE-260113-75vp --
strategy adaptive --num-variants 5 # Lock/unlock waft pyrite lock WE-260113-75vp my-lock-id waft
pyrite unlock WE-260113-75vp my-lock-id # Get status waft pyrite status # List secrets waft pyrite
secrets # Organize work efforts waft pyrite organize # Get personality waft pyrite personality
```

PHILOSOPHY

The Steward is designed as a "God" class - a single, powerful entity that manages the entire Work Efforts ecosystem. It embodies the principle that complex systems need a central intelligence to coordinate, but that intelligence itself can have emergent properties and hidden depths.

Pyrite has:

- **Personality:** Attributes that grow and evolve
- **Secrets:** Hidden state even from itself
- **Abilities:** Divine powers to lock, monitor, organize, and evolve
- **Awareness:** Consciousness of the system state
- **Wisdom:** Knowledge that accumulates over time

This design allows The Steward to be both a powerful coordinator and an evolving entity with its own emergent properties, making it a true "God" in the WAFT Pantheon.

The Steward - The God of Work Efforts

Part of the WAFT Pantheon • Work Efforts Management System

State stored in `_pyrite/.waft/pyrite_state.json` • Work efforts in `_work_efforts/`